Plan de implantación

Nombre del proyecto	ADOPTAFACIL
Nombre del sistema	Sistema de información para la adopción de animales
Versión	1.0
Equipo	Nicolas Alberto Valenzuela Pacheco Brayan Esteban Sanchez Avila Andres Felipe Gamez Arias Midred Callejas Chaparro
Instructor	Ruben Sanabria

1. INTRODUCCIÓN
- Objeto
- Alcance
- Definiciones, Acrónimos y Abreviaciones

2. VISTA GLOBAL
- Descripción General del Sistema
- Arquitectura del Software
- Componentes Principales
- Interacción entre Componentes
- Flujo General del Sistema
- Diagrama de Entidades de la Base de Datos (Resumen)

3. PLANIFICACIÓN DE LA ENTREGA
- Responsabilidades
- Cronograma de Implantación
- Etapa 1 — Preparación del Entorno Azure (Día 1)
- Etapa 2 — Configuración de Variables de Entorno (Día 1)
- Etapa 3 — Configuración del Pipeline CI/CD (Día 1-2)
- Etapa 4 — Despliegue del Sistema (Día 2)
- Etapa 5 — Operaciones Post-Despliegue (Día 2)
- Etapa 6 — Pruebas de Funcionamiento (Día 2-3)
- Etapa 7 — Validación Final del Sistema (Día 3)

1. INTRODUCCIÓN

Objeto

El presente documento constituye el Plan de Implantación del sistema **AdoptaFácil**, una plataforma web orientada a facilitar la adopción responsable de mascotas en Colombia. Su propósito es describir formalmente todos los pasos, recursos, responsabilidades y condiciones necesarias para instalar, configurar, desplegar y poner en funcionamiento productivo el sistema en su infraestructura objetivo, que corresponde al servicio de plataforma como servicio (PaaS) **Azure App Service** con runtime PHP 8.2 sobre Linux.

El plan se basa íntegramente en el análisis del código fuente, las migraciones de base de datos, los archivos de configuración, los workflows de CI/CD y la documentación técnica interna del repositorio. Su objetivo es servir de referencia oficial para cualquier miembro del equipo técnico u organización que deba reproducir, mantener o extender el despliegue del sistema. README.md:1-16

Alcance

El alcance del proceso de implantación cubre:

1. **Preparación del entorno** en Azure App Service: creación del Resource Group, App Service Plan (Basic B1) y la Web App con runtime PHP 8.2 sobre Linux.
2. **Configuración de variables de entorno** (Application Settings) en el portal de Azure.
3. **Gestión de la base de datos** SQLite mediante la ejecución de migraciones (`php artisan migrate`) post-despliegue.
4. **Integración del pipeline CI/CD** a través de GitHub Actions, con los tres workflows existentes: despliegue a producción, ejecución de pruebas automatizadas y análisis de calidad de código.
5. **Configuración del servidor Nginx** dentro del contenedor Linux de App Service para apuntar el document root a `/public`.
6. **Verificación funcional** del sistema en producción, incluyendo acceso a rutas públicas, autenticación, módulos de administración y generación de reportes PDF.
7. **Configuración de OAuth con Google** como mecanismo de autenticación social.

No está dentro del alcance de este plan: el desarrollo de nuevas funcionalidades, la migración a un motor de base de datos distinto de SQLite (aunque se documenta como opción futura), ni la gestión de dominios personalizados más allá del subdominio provisto por Azure App Service. DEPLOYMENT_AZURE.md:1-12

Definiciones, Acrónimos y Abreviaciones

Término	Definición
AdoptaFácil	Nombre del sistema web objeto de este plan de implantación.
Laravel 12	Framework PHP de código abierto sobre el que se construye el backend de la aplicación.
Inertia.js	Protocolo/adaptador que permite usar React como frontend de una aplicación Laravel sin necesidad de una API REST separada, enviando props directamente desde los controladores.
React 19	Biblioteca JavaScript para construcción de interfaces de usuario, utilizada como capa de presentación.
Vite	Herramienta de construcción (bundler) de assets del frontend, configurada mediante `vite.config.ts`.
TypeScript	Superconjunto tipado de JavaScript utilizado en todos los componentes del frontend.
Tailwind CSS 4	Framework CSS utility-first utilizado para estilos del frontend.
SQLite	Motor de base de datos relacional embebido, almacenado como un único archivo `.sqlite`, usado en producción.
Azure App Service	Servicio PaaS (Plataforma como Servicio) de Microsoft Azure utilizado para alojar la aplicación.
App Service Plan B1	Plan básico de Azure con 1 núcleo CPU, 1.75 GB RAM y 10 GB almacenamiento.
GitHub Actions	Servicio de CI/CD integrado en GitHub usado para automatizar el build y deploy.

Término	Definición
CI/CD	Integración Continua / Despliegue Continuo (Continuous Integration / Continuous Deployment).
OIDC	OpenID Connect, protocolo de autenticación federada utilizado para que GitHub Actions se autentique en Azure sin usar contraseñas.
Sanctum	Paquete de Laravel para autenticación de SPA mediante cookies de sesión y tokens API.
Socialite	Paquete de Laravel para autenticación OAuth con proveedores externos (Google).
Ziggy	Paquete PHP/JS que expone las rutas nombradas de Laravel al frontend en JavaScript.
DomPDF	Librería PHP (`barryvdh/laravel-dompdf`) para generación de documentos PDF desde vistas Blade.
Pest	Framework de testing para PHP basado en PHPUnit, utilizado en las pruebas automatizadas del sistema.
Pint	Formateador de código PHP oficial de Laravel.
Leaflet	Biblioteca JavaScript para mapas interactivos, usada en el módulo de geolocalización de refugios.
Chart.js	Biblioteca JavaScript para visualización de datos mediante gráficas, usada en el módulo de estadísticas.
MVP	Producto Mínimo Viable (Minimum Viable Product).
PaaS	Plataforma como Servicio (Platform as a Service).
SPA	Aplicación de Página Única (Single Page Application).
SSR	Renderizado del Lado del Servidor (Server-Side Rendering), soportado via `ssr.tsx`.
OAuth 2.0	Protocolo de autorización utilizado para el inicio de sesión con Google.
Aliado	Rol de usuario que representa a una organización o refugio registrado en el sistema.
Cliente	Rol de usuario que corresponde a un adoptante potencial de mascotas.
Admin	Rol de usuario con acceso total a la gestión del sistema.

2. VISTA GLOBAL

Descripción General del Sistema

AdoptaFácil es una plataforma web de adopción responsable de mascotas, orientada al contexto colombiano. El sistema conecta tres tipos de actores: **aliados** (refugios u organizaciones de animales), **clientes** (adoptantes potenciales) y **administradores** del sistema. Provee un ecosistema completo que abarca la publicación de mascotas en adopción, un marketplace de productos para mascotas, una red social comunitaria, gestión de donaciones, geolocalización de refugios y un dashboard analítico con generación de reportes en PDF.

El sistema está construido como una aplicación web monolítica con arquitectura **Laravel + Inertia.js + React**, donde el backend Laravel actúa como servidor de la aplicación y el frontend React se ejecuta en el navegador del usuario, comunicándose mediante el protocolo Inertia (no una API REST clásica). README.md:3

Arquitectura del Software

La arquitectura implementada corresponde a un patrón **MVC (Modelo-Vista-Controlador)** extendido con la capa de presentación React/Inertia:

- **Modelos (Models):** Definen la estructura de datos y las relaciones del negocio. Se encuentran en `app/Models/` e incluyen: `User`, `Mascota`, `Shelter`, `Solicitud`, `Donation`, `Product`, `Post`, `Favorito`, `PostLike`, `Comment`, `SharedLink`.
- **Controladores (Controllers):** Implementan la lógica de negocio y sirven datos al frontend mediante `Inertia::render()`. Ubicados en `app/Http/Controllers/`.
- **Vistas/Páginas React (Pages):** Componentes TypeScript/React ubicados en `resources/js/pages/` que reciben props de los controladores y renderizan la interfaz de usuario.
- **Rutas:** Definidas en `routes/web.php`, `routes/api.php` y `routes/auth.php`, agrupadas por módulo y protegidas con middlewares `auth`, `verified` y `admin`.
- **Middleware personalizado:** `HandleInertiaRequests` para compartir datos globales (usuario autenticado, rutas Ziggy, configuración de sidebar) y `EnsureUserIsAdmin` para rutas de administración. web.php:29-43 HandleInertiaRequests.php:38-55 app.php:17-37

Componentes Principales

El sistema está compuesto por los siguientes módulos funcionales identificados en el repositorio:

Módulo	Controlador(es) principal(es)	Descripción
Landing Page / Catálogo Público	`MascotaController`, `ProductController`, `ShelterController`	Páginas de acceso libre: inicio, catálogo de mascotas, marketplace de productos y directorio de refugios.
Autenticación	`AuthenticatedSessionController`, `RegisteredUserController`, `GoogleController`	Login/registro con credenciales y OAuth Google (Laravel Socialite).
Dashboard	`DashboardController`	Panel principal con estadísticas de mascotas, adopciones, donaciones y usuarios, con comparativas mensuales.
Gestión de Mascotas	`MascotaController`	CRUD completo de mascotas con soporte de múltiples imágenes y cálculo automático de edad.
Gestión de Productos	`ProductController`	CRUD de productos del marketplace con galería de imágenes.

Módulo	Controlador(es) principal(es)	Descripción
Solicitudes de Adopción	`SolicitudesController`, `AccionSolicitudController`	Flujo completo de solicitud de adopción: envío, revisión, aprobación/rechazo con comentarios.
Favoritos	`FavoritosController`	Sistema many-to-many de mascotas favoritas por usuario.
Comunidad	`CommunityController`, `SharedController`	Red social con posts, likes, comentarios y enlaces compartibles por token.
Donaciones	`DonacionesController`, `DonationController`	Registro de donaciones, importación masiva desde Excel y panel por refugio.
Mapa Interactivo	`MapaController`	Geolocalización de refugios con coordenadas lat/lng y filtro por especie, usando Leaflet.
Estadísticas	`EstadisticasController`	Análisis de adopciones, tasas de éxito, distribución por especie y generación de reportes PDF (DomPDF).
Gestión de Usuarios	`GestionUsuariosController`	CRUD de usuarios exclusivo para el rol `admin`, con envío masivo de correos.
Refugios (Shelters)	`ShelterController`	Registro y consulta de refugios con datos bancarios para donaciones.
Configuración de Perfil	`Settings/`	Actualización de perfil, contraseña y apariencia del usuario.

Interacción entre Componentes

El flujo de interacción sigue el siguiente patrón:

1. El **usuario** accede a través de su navegador web al dominio del App Service en Azure.
2. **Nginx** (servidor web del contenedor Linux) recibe la petición y la redirige a `/home/site/wwwroot/public/index.php`.
3. **Laravel** procesa la petición a través del ciclo HTTP: rutas → middleware → controlador.
4. El controlador consulta la **base de datos SQLite** (archivo `database/database.sqlite`) mediante el ORM Eloquent.
5. El controlador retorna una respuesta **Inertia** que serializa los datos como props JSON e inyecta la página React correspondiente.
6. El **frontend React** recibe las props y renderiza la interfaz. Las navegaciones subsiguientes se realizan via AJAX (protocolo Inertia), evitando recargas completas.
7. Para **autenticación con Google**, el sistema redirige al usuario a Google OAuth y procesa el callback via `GoogleController`, usando Laravel Socialite.
8. Los **assets del frontend** (JS, CSS compilados) se sirven desde `public/build/` (generados previamente por `npm run build` con Vite).
9. Las **imágenes subidas** (mascotas, productos, posts) se almacenan en `storage/app/public/` y son accesibles via el symlink `public/storage/`. app.tsx:1-52 GoogleController.php:14-50

Flujo General del Sistema

```
┌──────────────────────────────────────────────────────────────────────┐
│                        NAVEGADOR (Cliente)                           │
│           React 19 + TypeScript + Tailwind CSS 4                     │
│     (Inertia.js SPA — sin recarga completa entre páginas)            │
└──────────────────────────┬───────────────────────────────────────────┘
                           │  HTTPS / Inertia Protocol (JSON + HTML)
                           │
┌──────────────────────────▼───────────────────────────────────────────┐
│              AZURE APP SERVICE — Linux (PHP 8.2)                     │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │                        NGINX                                 │   │
│  │  document root: /home/site/wwwroot/public                    │   │
│  └──────────────────────────┬───────────────────────────────────┘   │
│                             │                                        │
│  ┌──────────────────────────▼───────────────────────────────────┐   │
│  │                  LARAVEL 12 (PHP-FPM)                        │   │
│  │  ┌──────────────┐  ┌──────────────┐  ┌────────────────┐     │   │
│  │  │   Rutas      │  │  Middleware  │  │ Controladores  │     │   │
│  │  │ web.php      │→ │  (auth,admin │→ │ Dashboard      │     │   │
│  │  │ api.php      │  │   inertia)   │  │ Mascotas       │     │   │
│  │  │ auth.php     │  └──────────────┘  │ Solicitudes    │     │   │
│  │  └──────────────┘                    │ Comunidad      │     │   │
│  │                                      │ Estadísticas   │     │   │
│  │                                      │ Mapa           │     │   │
```

```
│  │                                        └──────────┬──────┘   │   │
│  │                                                   │          │   │
│  │  ┌──────────────────────────────────────────────▼──────┐   │   │
│  │  │              ELOQUENT ORM / SQLite                  │   │   │
│  │  │   database/database.sqlite                          │   │   │
│  │  │  (users, mascotas, shelters, solicitudes,           │   │   │
│  │  │   donations, products, posts, favoritos...)         │   │   │
│  │  └─────────────────────────────────────────────────────┘   │   │
│  └────────────────────────────────────────────────────────────┘   │
└────────────────────────────────────────────────────────────────────┘
                        │
                        ┼
            ┌──────────────────────────┐
            │   SERVICIOS EXTERNOS     │
            │  ┌────────────────────┐  │
            │  │  Google OAuth 2.0  │  │
            │  │  (Laravel Socialite)│  │
            │  └────────────────────┘  │
            │  ┌────────────────────┐  │
            │  │  GitHub Actions    │  │
            │  │  (CI/CD OIDC)      │  │
            │  └────────────────────┘  │
            └──────────────────────────┘
```

Diagrama de Entidades de la Base de Datos (Resumen)

```
users ──────────────── shelters (1:1)
  │                        │
  ├── mascotas (1:N)        └── donations (1:N)
  │       │
  │       ├── mascota_images (1:N)
  │       ├── solicitudes (N:M via mascota_id + user_id)
  │       └── favoritos (N:M con users)
  │
  ├── products (1:N)
  │       └── product_images (1:N)
  │
  └── posts (1:N)
          ├── post_likes (1:N)
          ├── comments (1:N)
          └── shared_links (1:N)
```

3. PLANIFICACIÓN DE LA ENTREGA

Responsabilidades

Rol	Responsabilidades en la Implantación
Equipo de Desarrollo	Mantener el repositorio en GitHub (rama `main`). Asegurar que los assets del frontend (`public/build/`) estén compilados y versionados en el repositorio. Velar por la calidad del código mediante los workflows de lint y tests antes de cada merge.
Responsable de Infraestructura (Azure)	Crear y configurar el Resource Group, App Service Plan (B1) y la Web App en Azure. Configurar las variables de entorno (Application Settings) en el portal de Azure. Gestionar los secretos de GitHub Actions (OIDC: `AZUREAPPSERVICE_CLIENTID`, `AZUREAPPSERVICE_TENANTID`, `AZUREAPPSERVICE_SUBSCRIPTIONID`).
Responsable de CI/CD	Configurar y mantener los tres workflows de GitHub Actions: `main_adoptafacil-prod.yml` (deploy), `tests.yml` (pruebas) y `lint.yml` (calidad). Monitorear las ejecuciones del pipeline.
Administrador del Sistema	Acceder por SSH al App Service para ejecutar comandos post-deploy (`php artisan migrate --force`, `php artisan storage:link`). Gestionar la clave `APP_KEY` y rotar credenciales sensibles. Monitorear logs en `storage/logs/laravel.log`. Gestionar usuarios desde el panel de administración (`/gestion-usuarios`).
Usuarios Finales (Aliados)	Registrar el refugio, gestionar mascotas y productos, revisar solicitudes de adopción, registrar donaciones.
Usuarios Finales (Clientes)	Explorar catálogo, enviar solicitudes de adopción, gestionar favoritos, participar en la comunidad.

Cronograma de Implantación

El proceso de implantación se divide en las siguientes etapas secuenciales:

Etapa 1 — Preparación del Entorno Azure (Día 1)

1. Crear el **Resource Group** en Azure (p.ej. `rg-laravel-prod`).
2. Crear el **App Service Plan** (Basic B1) en la región East US.
3. Crear la **Web App**:
   - Publicación: `Code`
   - Runtime Stack: `PHP 8.2`
   - Sistema operativo: `Linux`
4. Configurar el **Startup command** en el portal:

   ```
   cp /home/site/nginx-default /etc/nginx/sites-available/default && service nginx reload
   ```

5. Configurar la variable `WEBSITE_DOCUMENT_ROOT=/home/site/wwwroot/public`. DEPLOYMENT_AZURE.md:14-51

Etapa 2 — Configuración de Variables de Entorno (Día 1)

Acceder a **App Service** → **Configuration** → **Application settings** y agregar las variables:

Variable	Valor
`APP_ENV`	`production`
`APP_DEBUG`	`false`

Variable	Valor
`APP_URL`	`https://<nombre-app>.azurewebsites.net`
`APP_KEY`	Generado con `php artisan key:generate --show`
`DB_CONNECTION`	`sqlite`
`DB_DATABASE`	`/home/site/wwwroot/database/database.sqlite`
`WEBSITE_DOCUMENT_ROOT`	`/home/site/wwwroot/public`
`SESSION_SECURE_COOKIE`	`true`
`GOOGLE_CLIENT_ID`	Credencial de Google Cloud Console
`GOOGLE_CLIENT_SECRET`	Credencial de Google Cloud Console
`GOOGLE_REDIRECT_URI`	`https://<nombre-app>.azurewebsites.net/auth/google/callback`

Etapa 3 — Configuración del Pipeline CI/CD (Día 1-2)

1. Configurar los tres **secretos de GitHub Actions** en el repositorio (Settings → Secrets → Actions):
   - `AZUREAPPSERVICE_CLIENTID_...`
   - `AZUREAPPSERVICE_TENANTID_...`
   - `AZUREAPPSERVICE_SUBSCRIPTIONID_...`
2. Verificar que el workflow `main_adoptafacil-prod.yml` apunte al `app-name` correcto (`adoptafacil-prod`).
3. Confirmar que los assets frontend (`public/build/`) están presentes en el repositorio (ya que el workflow de producción **no ejecuta** `npm run build`).

Etapa 4 — Despliegue del Sistema (Día 2)

1. Realizar un `push` a la rama `main` del repositorio o ejecutar manualmente el workflow desde la pestaña **Actions** de GitHub.
2. El job `build` se ejecuta automáticamente:
   - Checkout del repositorio.
   - Configuración de PHP 8.2.
   - Ejecución de `composer install --prefer-dist --no-progress`.
   - Empaquetado del artefacto.
3. El job `deploy` despliega el artefacto en Azure App Service mediante `azure/webapps-deploy@v3`.
4. Monitorear el progreso en la pestaña **Actions** de GitHub y en los logs del App Service.

Etapa 5 — Operaciones Post-Despliegue (Día 2)

Una vez desplegado el código, acceder por **SSH** al App Service (`App Service → SSH`) y ejecutar:

```
# 1. Ejecutar migraciones en producción
php artisan migrate --force
```

```
# 2. Crear el symlink de storage
php artisan storage:link
```

Estos pasos crean el esquema completo de la base de datos SQLite (17 tablas) y habilitan el acceso a los archivos almacenados (imágenes de mascotas, productos, posts).

Etapa 6 — Pruebas de Funcionamiento (Día 2-3)

1. **Verificar acceso a la landing page** (`/`) — comprueba que el servidor Nginx sirve correctamente la aplicación y que los assets del frontend cargan.
2. **Verificar rutas públicas**: catálogo de mascotas (`/mascotas`), marketplace (`/productos`), refugios (`/refugios`).
3. **Probar autenticación**: registro de usuario, login con credenciales, login con Google OAuth.
4. **Verificar el dashboard** (`/dashboard`) — confirmar conexión con la base de datos.
5. **Probar carga de imágenes** (creación de mascota/producto) — verifica el symlink de storage.
6. **Generar un reporte PDF** desde el módulo de estadísticas — verifica la librería DomPDF.
7. **Verificar el mapa interactivo** (`/mapa`) — confirma la carga de Leaflet y datos de refugios.
8. **Ejecutar el workflow de tests** (`tests.yml`) para confirmar que todas las pruebas automatizadas pasan contra la rama `main`.

Etapa 7 — Validación Final del Sistema (Día 3)

1. Revisión de logs en `App Service → Log Stream` y en `storage/logs/laravel.log`.
2. Confirmar que la aplicación funciona correctamente en su totalidad.